

NATIONAL ECONOMICS UNIVERSITY (NEU)
FACULTY OF DATA SCIENCE & FINANCIAL ENGINEERING (DSFE)

MIDTERM PROGRESS EXAM: PART 01

Course: Fundamental Programming Concepts in Python (DSAI1003)

Scope: Computers, Python Architecture, Error Diagnosis, Algorithms & Basic I/O

Full Name: _____ **Student ID:** _____

Class / Cohort: _____ **Exam Room / Desk:** _____

Exam Duration: 60 minutes

Maximum Marks: 100 points

Important Instructions for Students:

- This is a closed-book examination. All electronic devices and internet access are strictly prohibited.
- The exam contains 4 sections across 7 pages. Total allocated time is 60 minutes.
- In Section D (Python Programming), write standard statements adhering to the **Input → Process → Output** pattern. Do **not** define custom functions (def) or use loops/branching.

Section A: Conceptual & Architectural Fundamentals

(20 points)

Answer all 10 questions. Each correct answer carries 2 points. Suggested time: 12 minutes.

Q1. Which computer hardware component is directly responsible for fetching, decoding, and executing program instructions?

- A. Primary Memory (RAM)
- B. Central Processing Unit (CPU)
- C. Secondary Storage (SSD / Hard Disk)
- D. Input/Output Bus

Your choice: _____

Q2. What happens to the information stored in **Primary Memory (RAM)** when the computer is shut down?

- A. It is permanently stored in memory cells.
- B. It is lost because RAM is volatile memory.
- C. It is automatically compressed and transferred to cache.
- D. It is rewritten to CPU registers for immediate restart.

Your choice: _____

Q3. Why can a single physical computer perform thousands of completely different tasks?

- A. The physical CPU hardware rewires itself automatically for each task.
- B. It can load and execute different software programs consisting of instructions and decisions.
- C. High-level languages bypass memory and directly command peripheral circuits.

D. Secondary storage replaces CPU registers during complex computation.

Your choice: _____

Q4. Which sequence accurately describes the execution pipeline of a standard Python program?

- A. Source Code (.py) → Machine Code → Compiler → Execution
- B. Source Code (.py) → Byte Code (.pyc) → Python Virtual Machine (PVM) → Running Program
- C. Byte Code (.pyc) → Source Code (.py) → Operating System → CPU
- D. Source Code (.py) → Linker → Binary File (.exe) → CPU

Your choice: _____

Q5. Which of the following statements regarding high-level languages like Python is **FALSE**?

- A. Python code is portable across Windows, macOS, and Linux operating systems.
- B. The CPU directly executes Python source code text without needing an interpreter or virtual machine.
- C. High-level languages allow programmers to express algorithms using human-readable syntax.
- D. High-level languages free programmers from manually managing CPU registers and memory addresses.

Your choice: _____

Q6. What are the **three essential characteristics** that every valid algorithm must satisfy?

- A. Fast, Portable, and Object-Oriented
- B. Unambiguous, Executable, and Terminating
- C. Compiled, Abstract, and Infinite
- D. Linear, Mathematical, and Formatted

Your choice: _____

Q7. What is the main objective of creating **pseudocode** before writing Python code?

- A. To enable the Python interpreter to pre-compile variables into bytecode.
- B. To design and verify the algorithmic logic in plain language without getting bogged down by syntax details.
- C. To eliminate the need for primary memory during program execution.
- D. To generate automated unit tests for software applications.

Your choice: _____

Q8. What data type does the built-in `input()` function **always** return in Python?

- A. `int`
- B. `float`
- C. `str`
- D. Depends automatically on whether the input text contains numeric digits.

Your choice: _____

Q9. According to good programming practice, which sequence of problem-solving steps is most appropriate?

- A. Write Code → Run Program → Understand Problem → Pseudocode
- B. Understand Problem → Develop Algorithm/Pseudocode → Test by Hand → Translate to Python → Run & Debug
- C. Design Algorithm → Understand Problem → Write Code → Test by Hand
- D. Write Code → Reverse-engineer Pseudocode → Debug

Your choice: _____

Q10. Which of the following is the best example of a **Logic Error** (semantic error)?

- A. Omitting a closing parenthesis: `print("Hello".`
- B. Attempting to divide a number by zero: `print(25 / 0).`
- C. Calculating rectangle perimeter as `perimeter = length * width` instead of `2 * (length + width).`
- D. Attempting to convert alphabetical letters to an integer: `int("twenty").`

Your choice: _____

Section B: Code Tracing, Output Prediction & Error Diagnosis

(25 points)

Answer all 5 questions. Each question carries 5 points. Suggested time: 15 minutes.

Q11. Precise Output Prediction: sep and end (5 points)

Carefully analyze the Python snippet below:

```
print("2026", "09", "10", sep="-", end=" ")
print("EXAM", "PYTHON", sep="::")
print("DONE")
```

Write the **exact screen output** below. Clearly show any spaces and newlines:

Output Box for Q11

Q12. String Operations vs. Numeric Arithmetic (5 points)

Given the following variable assignments and statements:

```
a = "5"
b = "2"
c = 5
d = 2

print(a + b)
print(c + d)
print(a * 3)
print(c * 3)
```

Provide the exact printed output for each line:

- Output line 1 (print(a + b)): _____
- Output line 2 (print(c + d)): _____
- Output line 3 (print(a * 3)): _____
- Output line 4 (print(c * 3)): _____

Q13. Error Taxonomy and Classification (5 points)

For each code segment, classify the error into **Compile-Time / Syntax Error**, **Exception (Run-time Error)**, or **Logic Error**. Provide a 1-sentence technical explanation.

Code Snippet	Error Classification	Technical Reason
A. print("Total is:" total)		
B. val = int("3.14159")		
C. avg = score1 + score2 + score3 / 3 (Goal: calculate mean of 3 scores)		

Q14. Deconstructing Python Statements (5 points)

Examine the following statement:

```
# Display computed final score with bonus  
print("Final Score:", base_score + 10)
```

Identify each corresponding element from the statement:

- The **comment**: _____
- The **built-in function name**: _____
- The **string literal argument**: _____
- The **arithmetic expression argument**: _____
- The **variable name**: _____

Q15. Tracing Formatted f-Strings (5 points)

Predict the exact screen output produced by the following snippet:

```
item = "USB Drive"  
quantity = 4  
unit_price = 12.5  
subtotal = quantity * unit_price  
  
print(f"Item: {item} | Qty: {quantity}")  
print(f"Subtotal: ${subtotal:.2f}")  
print(f"Discount (10%): ${subtotal * 0.10:.2f}")
```

Output Box for Q15

Section C: Algorithm Design & Pseudocode

(25 points)

Answer all tasks. Suggested time: 15 minutes.

Problem Specification: Electric Scooter Rental Billing System

A city electric scooter operator calculates trip fares based on the following rules:

- **Unlock Fee:** \$1.50 fixed rate per trip.
- **Ride Time Rate:** \$0.25 for each minute ridden.
- **Sales Tax:** An 8% (0.08) municipal sales tax is added onto the subtotal fare (Unlock Fee + Ride Cost).

Your task is to design an algorithm that computes and outputs an itemized customer receipt.

Task 1: Identify Inputs and Outputs (5 points)

- **Required Input(s):** _____
- **Required Output(s):** _____

Task 2: Write Structured Pseudocode (10 points)

Write clear, sequential pseudocode adhering to the **Input** → **Process** → **Output** paradigm:

Pseudocode Solution

Task 3: Algorithm Verification (5 points)

Demonstrate how your algorithm fulfills the three mandatory algorithm characteristics:

- Unambiguous:** _____
- Executable:** _____
- Terminating:** _____

Task 4: Hand-Trace Calculation (5 points)

Verify your algorithm manually for a trip duration of **20 minutes**:

- **Unlock Fee:** _____ **Riding Charge** ($20 \times \$0.25$): _____
- **Subtotal Before Tax:** _____ **Tax Amount** (8% of subtotal): _____
- **Final Total Fare to Pay:** _____

Section D: Applied Programming in Python

(30 points)

Write complete, syntactically correct Python statements directly using the **Input** → **Process** → **Output** pattern. Do **not** define custom functions (def), import, or loops. Suggested time: 18 minutes.

Problem 1: Commuter Fuel Cost Calculator (15 points)

Write a Python script that computes the fuel consumed and total monetary expense for a highway journey.

- Formulas:

$$\text{fuel_used} = \frac{\text{distance} \times \text{consumption_rate}}{100}, \quad \text{trip_cost} = \text{fuel_used} \times \text{fuel_price}$$

- Requirements:
 - Prompt user for distance in km, consumption_rate (L/100km), and fuel_price per liter using `input()`. Ensure proper type conversion to float.
 - Compute `fuel_used` and `trip_cost`.
 - Display the summary using f-strings with values formatted to **2 decimal places** (`:.2f`).

Python Solution for Problem 1

Problem 2: Event Duration Breakdown (15 points)

Write a Python script that converts a total number of elapsed seconds into **hours, minutes, and remaining seconds** using integer floor division (`//`) and modulus (`%`).

- Requirements:
 - Prompt user for `total_seconds` and cast to `int`.
 - Compute full hours (1 hr = 3600 s), remaining seconds, full minutes (1 min = 60 s), and leftover seconds.
 - Display the result in the format: "X hours, Y minutes, Z seconds".

Python Solution for Problem 2

— END OF EXAM PAPER —